1) Calculator Application:
a) Using with and without R objects on console.

1. Without `with()` function

```
a <- 10
b <- 5

result1 <- a + b
result2 <- a - b
result3 <- a * b
result4 <- a / b

print(result1)
print(result2)
print(result3)
print(result4)
```

2. With `with()` function

```
calculator <- list(a = 10, b = 5)

with(calculator, {
  print(a + b)
  print(a - b)
  print(a * b)
  print(a / b)
})
```

(b) Using mathematical functions on the R console

```
x <- 25

print(sqrt(x))
print(abs(-10))
print(log(x))
print(exp(2))
print(round(3.567, 2))
print(ceiling(4.2))
print(floor(4.8))
print(sum(10, 20, 30))
print(max(10, 20, 30))
```

```
print(min(10, 20, 30))
```

(c) R script to create calculator objects and save them to a specified disk location

Program: `calculator.R`

```
a <- 20
b <- 10

add <- a + b
subtract <- a - b
multiply <- a * b
divide <- a / b

calculator <- list(
  a = a,
  b = b,
  addition = add,
  subtraction = subtract,
  multiplication = multiply,
  division = divide
)

print(calculator)

save(calculator, file = "D:/calculator.RData")
```

(d) R Packages: tidyr, ggplot2, ggraph, glue and Shiny

1. `tidyr` – Organizing data

```
library(tidyr)

data <- data.frame(
  Name = c("A", "B"),
  Math = c(90, 80),
  Science = c(85, 75)
)
```

```
result <- pivot_longer(
  data,
  cols = c(Math, Science),
  names_to = "Subject",
  values_to = "Marks"
)
print(result)
```

2. `ggplot2` – Drawing a graph

```
library(ggplot2)

data <- data.frame(
  Name = c("A", "B", "C"),
  Marks = c(80, 90, 70)
)

ggplot(data, aes(x = Name, y = Marks)) +
  geom_bar(stat = "identity")
```

3. `ggraph` – Drawing graph networks

```
library(tidygraph)
library(ggraph)

edges <- data.frame(
  from = c("A", "B"),
  to = c("B", "C")
)

graph <- as_tbl_graph(edges)

ggraph(graph, layout = "circle") +
  geom_edge_link() +
  geom_node_point(size = 5) +
  geom_node_text(aes(label = name), vjust = 2)
```

4. `glue` – Combining text and variables

```
library(glue)

a <- 10
```

```
b <- 5

result <- a + b

print(glue("Addition of {a} and {b} is {result}"))
```

5. Shiny Program in R

```
library(shiny)

ui <- fluidPage(
  titlePanel("Calculator"),
  numericInput("a", "Enter first number:", 10),
  numericInput("b", "Enter second number:", 5),
  textOutput("result")
)

server <- function(input, output) {
  output$result <- renderText({
    input$a + input$b
  })
}

shinyApp(ui, server)
```

2) Descriptive Statistics – R Programs

(a) Basic descriptive statistics using `summary()`

```
data <- c(10, 20, 30, 40, 50)

summary(data)
```

(b) Find a subset of a dataset using `subset()`

```
data <- data.frame(
  Name = c("A", "B", "C", "D"),
  Marks = c(80, 45, 90, 60)
)

result <- subset(data, Marks > 60)
```

```
print(result)
```

3) Reading and Writing Different Types of Datasets – R Programs

(a) Reading and writing `.txt` and `.csv` files

1. Reading a TXT file from disk

```
data <- read.table("D:/data.txt", header = TRUE)
print(data)
```

2. Reading a CSV file from disk

```
data <- read.csv("D:/data.csv")
print(data)
```

3. Reading a CSV file from the web

```
url <- "https://example.com/data.csv"
data <- read.csv(url)
print(data)
```

4. Writing data to a specific disk location

```
data <- data.frame(
  Name = c("A", "B", "C"),
  Marks = c(80, 90, 70)
)

write.csv(data, "D:/output.csv", row.names = FALSE)
```

(b) Reading an Excel data sheet in R

```
library(readxl)

data <- read_excel("D:/data.xlsx")
print(data)
```

To install the package once:

```
install.packages("readxl")
```

(c) Reading an XML dataset in R

```
library(xml2)

data <- read_xml("D:/data.xml")
print(data)
```

4) Visualizations – R Programs

(a) Find data distributions using box plot and scatter plot

1. Box plot

```
marks <- c(40, 50, 60, 70, 80, 90, 100)

boxplot(marks, main = "Box Plot of Marks",
        col = "lightblue")
```

2. Scatter plot

```
x <- c(1, 2, 3, 4, 5)
y <- c(10, 20, 15, 30, 25)

plot(x, y, main = "Scatter Plot",
     xlab = "X", ylab = "Y",
     col = "blue", pch = 19)
```

(b) Find outliers using plot

```
data <- c(10, 12, 15, 18, 20, 22, 100)

boxplot(data, main = "Outlier Detection",
        col = "lightgreen")

boxplot.stats(data)$out
```

(c) Histogram, bar chart and pie chart on sample data

1. Histogram

```
marks <- c(40, 50, 60, 60, 70, 80, 90)

hist(marks, main = "Histogram of Marks",
     xlab = "Marks", col = "skyblue")
```

2. Bar chart

```
marks <- c(80, 90, 70)
names(marks) <- c("A", "B", "C")

barplot(marks, main = "Students Marks",
        col = "orange")
```

3. Pie chart

```
marks <- c(80, 90, 70)
names(marks) <- c("A", "B", "C")

pie(marks, main = "Students Marks",
    col = c("red", "green", "blue"))
```

5) Correlation and Covariance – R Programs

(a) Find the correlation matrix on Iris data

```
data(iris)

result <- cor(iris[, 1:4])

print(result)
```

(b) Plot the correlation plot on Iris data

```
data(iris)

result <- cor(iris[, 1:4])

heatmap(result, main = "Correlation Plot")
```

(c) Analysis of variance (ANOVA) on Iris data

```
data(iris)

model <- aov(Sepal.Length ~ Species, data = iris)

summary(model)
```

6) R Program to Detect File Type and Read Accordingly

```
library(readxl)
library(xml2)

file <- "D:/data.csv"
ext <- tools::file_ext(file)

if (ext == "csv") {
  data <- read.csv(file)
} else if (ext == "txt") {
  data <- read.table(file, header = TRUE)
} else if (ext == "xlsx") {
  data <- read_excel(file)
} else if (ext == "xml") {
  data <- read_xml(file)
} else {
  print("Unsupported file type")
}

print(data)
```

Lab Cycle 2

1) Regression Model – Easy R Programs

(a) Logistic Regression for Student Admission

```
require(foreign)
require(MASS)

data <- read.csv("https://stats.idre.ucla.edu/stat/data/binary.csv")

data$rank <- factor(data$rank)

model <- glm(admit ~ gre + gpa + rank,
             data = data, family = binomial)

summary(model)

anova(model, test = "Chisq")
```

(b) Linear Regression to Predict Data

```
data(cars)

model <- lm(dist ~ speed, data = cars)

summary(model)

predict(model, data.frame(speed = 20))
```

2) Multiple Regression Model – Easy R Programs

(a) Develop a Multiple Regression Model

```
data(mtcars)

model <- lm(mpg ~ wt + hp + disp, data = mtcars)

summary(model)
```

(b) Apply Multiple Regression Model to Predict Data

```
data(mtcars)

model <- lm(mpg ~ wt + hp + disp, data = mtcars)

newdata <- data.frame(wt = 3, hp = 110, disp = 160)

predict(model, newdata)
```

3) Classification Model – Easy R Programs

(a) Install relevant packages for classification

```
install.packages("class")
install.packages("e1071")
install.packages("rpart")
```

(b) Implement and evaluate different classifiers

1. KNN Classifier using Iris dataset

```
library(class)
data(iris)
set.seed(123)

index <- sample(1:nrow(iris), 105)

train <- iris[index, 1:4]
test <- iris[-index, 1:4]

train_labels <- iris$Species[index]
test_labels <- iris$Species[-index]

result <- knn(train, test, train_labels, k = 3)

print(result)

accuracy <- mean(result == test_labels)
print(accuracy)
```

2. Naive Bayes Classifier using Iris dataset

```
library(e1071)
data(iris)

model <- naiveBayes(Species ~ ., data = iris)

result <- predict(model, iris)

print(result)

accuracy <- mean(result == iris$Species)
print(accuracy)
```

3. Decision Tree Classifier using Iris dataset

```
library(rpart)
data(iris)

model <- rpart(Species ~ ., data = iris, method = "class")

result <- predict(model, iris, type = "class")
```

```
print(result)

accuracy <- mean(result == iris$Species)
print(accuracy)
```

4) Clustering Model – R Programs

(a) Clustering algorithm for unsupervised classification

K-Means Clustering

```
data(iris)

x <- iris[, 1:4]

model <- kmeans(x, centers = 3)
print(model$cluster)
print(model$centers)
```

(b) Plot cluster data using R visualization

```
data(iris)

x <- iris[, 1:4]

model <- kmeans(x, centers = 3)

plot(x$Sepal. Length, x$Petal.Length,

    col = model$cluster,
    pch = 19,
    main = "K-Means Clustering",
    xlab = "Sepal Length",
    ylab = "Petal Length")
```

5) Time Series Model – R Programs

(a) Plot time series data using R visualization

```
data(AirPassengers)

plot(AirPassengers,
    main = "Air Passengers Time Series",
    xlab = "Year",
    ylab = "Number of Passengers",
    col = "blue",
    type = "l")
```

(b) Time Series Forecasting using Exponential Smoothing and ARIMA

1. Exponential Smoothing

```
data(AirPassengers)

model <- HoltWinters(AirPassengers)

plot(model)

forecast <- predict(model, n.ahead = 12)

print(forecast)
```

2. ARIMA Model

```
data(AirPassengers)

model <- arima(AirPassengers,
               order = c(1, 1, 1))

forecast <- predict(model, n.ahead = 12)

print(forecast$pred)
```

6) Data Analysis on `airquality` Dataset in R

```
data(airquality)

print(head(airquality))
print(summary(airquality))

data <- na.omit(airquality)

print(data)

hist(data$Temp, main="Temperature", col="blue")
boxplot(data$Ozone, main="Ozone Levels", col="green")


plot(data$Temp, data$Ozone,
    main="Temperature vs Ozone",
    xlab="Temperature", ylab="Ozone",
    col="red")

print(cor(data$Temp, data$Ozone))
```